

UNIVERSIDAD TECNOLÓGICA NACIONAL
TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

TRABAJO PRÁCTICO INTEGRADOR

**Sistema de Gestión de Categorías, Productos, Usuarios y Pedidos – FOOD
STORE**

Materia: Programación II

Alumno: Guzmán Mauricio Gabriel

Comisión: A25 C2-07 – Cursado a Distancia

Fecha de Entrega: 22 de junio de 2026

Universidad Tecnológica Nacional

Tabla de contenido

Marco Teórico.....	3
Decisiones Técnicas y Arquitectura.....	4
Menú de cada entidad creada (Capturas)	5
Categorías, productos, Usuarios y pedidos listados.....	6
Edición de pedidos mediante a la selección numérica de Estado y Forma de Pago	7
Dificultades y Soluciones	7
Referencias	9

Marco Teórico

El presente proyecto fue desarrollado utilizando el lenguaje de programación Java, aplicando los principios fundamentales de la Programación Orientada a Objetos (POO). Este paradigma permite modelar entidades del mundo real mediante clases y objetos, favoreciendo la reutilización, organización y mantenimiento del código.

Durante el desarrollo se aplicaron conceptos de encapsulamiento mediante el uso de atributos privados y métodos de acceso, herencia a través de una clase base común para las entidades del sistema, abstracción mediante la separación de responsabilidades en distintas capas e interfaces, y polimorfismo mediante la implementación de comportamientos comunes reutilizables.

La aplicación consiste en un sistema de gestión de categorías, productos, usuarios y pedidos, implementando las operaciones CRUD (Create, Read, Update y Delete) solicitadas en las historias de usuario. Cada módulo permite crear, listar, modificar y eliminar registros mediante una interfaz de consola.

Para el almacenamiento de datos se utilizaron estructuras dinámicas ArrayList, manteniendo la información en memoria durante la ejecución del programa. Aunque no se implementó persistencia en base de datos, la solución permite comprender el funcionamiento de las operaciones CRUD y la administración de relaciones entre entidades.

Dentro del proyecto se desarrollaron las entidades principales Categoría, Producto, Usuario, Pedido y DetallePedido. Además, se incorporó una clase Base de la cual heredan las demás entidades, permitiendo reutilizar atributos comunes como el identificador único y el indicador de eliminación lógica.

También se implementaron interfaces para representar comportamientos específicos del sistema. La interfaz Calculable fue utilizada en la entidad Pedido para calcular automáticamente el total de una compra a partir de los subtotales de cada detalle.

Se utilizaron enumeraciones (Enums) para representar conjuntos de valores predefinidos, mejorando la legibilidad y reduciendo errores de carga de datos. Entre ellas se encuentran Estado, FormaPago y Rol.

Por otra parte, se desarrollaron excepciones personalizadas para controlar errores específicos del negocio, tales como entidades inexistentes, correos electrónicos duplicados, precios

inválidos, stock insuficiente y datos incorrectos. Esto permitió mejorar la robustez de la aplicación y brindar mensajes claros al usuario.

Finalmente, se implementó el concepto de baja lógica mediante el atributo eliminado, evitando la eliminación física de los registros y conservando la información histórica, tal como fue requerido en las historias de usuario del proyecto.

Decisiones Técnicas y Arquitectura

Para abordar la solución se decidió implementar una arquitectura organizada por capas, con el objetivo de separar responsabilidades y facilitar el mantenimiento del código.

La aplicación fue estructurada en distintos paquetes:

Entities: contiene las clases que representan las entidades principales del sistema, tales como Categoría, Producto, Usuario, Pedido y DetallePedido.

Services: contiene la lógica de negocio y las operaciones CRUD. Se desarrollaron las clases CategoríaService, ProductoService, UsuarioService y PedidoService. Esta fue una de las decisiones técnicas más importantes del proyecto, ya que permitió separar completamente la lógica de negocio de la interfaz de usuario.

Inicialmente resultó complejo comprender la necesidad de esta capa, ya que muchas operaciones podían resolverse directamente desde los menús. Sin embargo, se optó por implementar servicios para centralizar validaciones, búsquedas, modificaciones, bajas lógicas y reglas de negocio, logrando una estructura más organizada y profesional.

Menu: contiene los menús utilizados para interactuar con el usuario mediante consola. Se implementó una clase abstracta denominada MenuBase para reutilizar la lógica de validación de opciones ingresadas por teclado.

Interfaces: incluye las interfaces utilizadas por el sistema, como Calculable y MenuPantalla.

Enums: agrupa las enumeraciones utilizadas para representar valores constantes del sistema.

Exception: contiene todas las excepciones personalizadas utilizadas para el manejo de errores.

La información se almacena temporalmente en memoria utilizando ArrayList, permitiendo gestionar las entidades durante la ejecución del programa. Cada servicio administra su propia colección de objetos y genera automáticamente identificadores únicos para los registros creados.

Respecto a los pedidos, se decidió implementar una relación entre Pedido y DetallePedido para permitir que un pedido contenga múltiples productos. Cada detalle calcula automáticamente su subtotal y posteriormente el pedido calcula el total general mediante la interfaz Calculable.

Además, se implementó baja lógica en todas las entidades principales, permitiendo mantener el historial de información sin eliminar físicamente los objetos de las colecciones.

Las capturas de pantalla incluidas en este informe muestran el funcionamiento de los distintos módulos del sistema, incluyendo la creación, modificación, listado y eliminación lógica de categorías, productos, usuarios y pedidos, así como las validaciones implementadas.

Menú de cada entidad creada (Capturas)

```
=== MENÚ PRINCIPAL ===  
1. Categorías  
2. Productos  
3. Usuarios  
4. Pedidos  
0. Salir  
Ingrese una opción: |
```

```
--- MENÚ CATEGORÍAS ---  
1. Crear categoría  
2. Listar categorías  
3. Editar categoría  
4. Eliminar categoría  
0. Volver  
Ingrese una opción: 2
```

```
--- MENÚ PRODUCTOS ---  
1. Crear producto  
2. Listar productos  
3. Editar producto  
4. Eliminar producto  
0. Volver  
Ingrese una opción: 0
```

```
--- MENÚ USUARIOS ---  
1. Crear usuario  
2. Listar usuarios  
3. Editar usuario  
4. Eliminar usuario  
0. Volver  
Ingrese una opción: 2  
⚠ No hay usuarios cargados
```

```

--- MENÚ PEDIDOS ---
1. Crear pedido
2. Listar pedidos
3. Editar pedido (estado/forma de pago)
4. Eliminar pedido
0. Volver
Ingrese una opción: 0

```

Categorías, productos, Usuarios y pedidos listados (Capturas)

```

--- MENÚ PRODUCTOS ---
1. Crear producto
2. Listar productos
3. Editar producto
4. Eliminar producto
0. Volver
Ingrese una opción: 2
Producto #1: Coca Cola 500ml ($2500,00) | Stock: 10 | Categoría: Bebidas frias
Producto #2: Sprite 500 ml ($2100,00) | Stock: 10 | Categoría: Bebidas frias

```

```

--- MENÚ CATEGORÍAS ---
1. Crear categoría
2. Listar categorías
3. Editar categoría
4. Eliminar categoría
0. Volver
Ingrese una opción: 2
Categoría #1: Bebidas | Gaseosas, jugos y aguas | Cantidad de productos: 0
Categoría #2: Hamburguesas | simples, dobles y triples | Cantidad de productos: 0

```

```

--- MENÚ PEDIDOS ---
1. Crear pedido
2. Listar pedidos
3. Editar pedido (estado/forma de pago)
4. Eliminar pedido
0. Volver
Ingrese una opción: 2
Pedido #1 | Usuario: Juan pablo Perez | Fecha: 2026-06-22 | Estado: PENDIENTE | FormaPago: EFECTIVO
-----
- DetallePedido #1: Coca cola Zero 500 ml x 2 => Subtotal: $5200,00
TOTAL DEL PEDIDO: $5200,00
-----

Pedido #2 | Usuario: Juan pablo Perez | Fecha: 2026-06-22 | Estado: PENDIENTE | FormaPago: EFECTIVO
-----
- DetallePedido #1: Fanta 500 ml x 2 => Subtotal: $4000,00
- DetallePedido #2: Coca cola Zero 500 ml x 1 => Subtotal: $2600,00
TOTAL DEL PEDIDO: $6600,00
-----

```

Edición de pedidos mediante a la selección numérica de Estado y Forma de Pago (Capturas)

```
--- MENÚ PEDIDOS ---
1. Crear pedido
2. Listar pedidos
3. Editar pedido (estado/forma de pago)
4. Eliminar pedido
0. Volver
Ingrese una opción: 3
ID del pedido a editar: 1

Seleccione nuevo estado:
1. PENDIENTE
2. CONFIRMADO
3. TERMINADO
4. CANCELADO
Ingrese una opción: 2

Seleccione nueva forma de pago:
1. TARJETA
2. TRANSFERENCIA
3. EFECTIVO
Ingrese una opción: 3
✅ Pedido actualizado correctamente.
```

Dificultades y Soluciones

Durante el desarrollo del proyecto surgieron diversas dificultades técnicas que permitieron profundizar los conocimientos adquiridos en la materia.

Una de las principales dificultades fue comprender cómo organizar correctamente la aplicación utilizando una arquitectura por capas. Particularmente, la implementación de los Services representó un desafío importante, ya que inicialmente no resultaba evidente por qué separar la lógica de negocio de los menús. Luego de investigar y realizar varias pruebas, se comprendió que esta estructura permitía centralizar las validaciones y operaciones CRUD, obteniendo un código más limpio, reutilizable y fácil de mantener.

Otro desafío fue la implementación de las relaciones entre entidades. Fue necesario comprender cómo asociar correctamente productos con categorías, usuarios con pedidos y pedidos con detalles, manteniendo la coherencia de la información en memoria.

También se presentaron dificultades relacionadas con las validaciones solicitadas por las historias de usuario. Se implementaron controles para evitar categorías duplicadas, correos electrónicos repetidos, precios negativos, stock negativo, cantidades inválidas y referencias a entidades inexistentes. Para resolver estas situaciones se desarrollaron excepciones personalizadas y se incorporó el manejo de errores mediante bloques try-catch.

La implementación de las bajas lógicas fue otro aspecto que requirió especial atención. Se necesitaba que los registros eliminados dejaran de aparecer en los listados sin ser eliminados físicamente de las colecciones. Esto se resolvió incorporando el atributo eliminado en la clase base y filtrando los registros al momento de mostrarlos.

En la gestión de pedidos surgieron dificultades para garantizar la integridad de los datos. Inicialmente podían generarse pedidos inconsistentes cuando ocurrían errores durante la carga de productos. Para solucionar este problema se implementó el manejo adecuado de excepciones y la cancelación de pedidos cuando las validaciones no eran superadas, evitando la creación de registros inválidos.

Finalmente, durante las pruebas se detectaron inconvenientes con la visualización de emojis y caracteres especiales en la consola integrada de NetBeans. Como solución se decidió ejecutar la aplicación desde PowerShell, logrando una correcta representación visual de los mensajes mostrados por el sistema.

En conclusión, los desafíos encontrados permitieron fortalecer conocimientos relacionados con Programación Orientada a Objetos, manejo de excepciones, organización de proyectos Java, arquitectura por capas y desarrollo de aplicaciones basadas en historias de usuario.

Referencias

Universidad Tecnológica Nacional. (2026). *Programación orientada a objetos 2: Profundización de temas* [Material de cátedra]. Tecnicatura Universitaria en Programación. URL: <https://tup.sied.utn.edu.ar/mod/resource/view.php?id=12212>

Universidad Tecnológica Nacional. (2026). *UML básico: Unidad dedicada al Lenguaje Unificado de Modelo y su relación con la Programación Orientada a Objetos* [Material de cátedra]. Tecnicatura Universitaria en Programación. URL: <https://tup.sied.utn.edu.ar/mod/resource/view.php?id=12233>

Universidad Tecnológica Nacional. (2026). *Programación II: Herencia y polimorfismo* [Material de cátedra]. Tecnicatura Universitaria en Programación. URL: <https://tup.sied.utn.edu.ar/mod/resource/view.php?id=12275>

Universidad Tecnológica Nacional. (2026). *Programación II - Colecciones* [Material de cátedra]. Tecnicatura Universitaria en Programación. URL: <https://tup.sied.utn.edu.ar/mod/resource/view.php?id=12255>

Universidad Tecnológica Nacional. (2026). *Programación II: Interfaces y manejo de errores* [Material de cátedra]. Tecnicatura Universitaria en Programación. URL: <https://tup.sied.utn.edu.ar/mod/resource/view.php?id=12294>

Link de video Trabajo Practico Integrador – Programación 2. Youtube: <https://youtu.be/hXi0bXyVrmA>

Link de Repositorio de GitHub: <https://github.com/MauricioGuzman29/Trabajo-Practico-Integrador---Programacion-2.-Food-Store-.git>

